

```
In [1]: import pandas as pd
import numpy as np

soil = pd.read_csv("soil_data.csv")

soil.head()
soil.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000000 entries, 0 to 999999
Data columns (total 17 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Soil_ID                               1000000 non-null int64
1   Location_Latitude                     1000000 non-null float64
2   Location_Longitude                    1000000 non-null float64
3   Depth_cm                             1000000 non-null float64
4   Texture                               1000000 non-null object
5   pH                                     1000000 non-null float64
6   Organic_Matter_%                      1000000 non-null float64
7   Moisture_Content_%                    1000000 non-null float64
8   Bulk_Density_g/cm³                    1000000 non-null float64
9   Nitrogen_N_ppm                        1000000 non-null float64
10  Phosphorus_P_ppm                      1000000 non-null float64
11  Potassium_K_ppm                       1000000 non-null float64
12  Cation_Exchange_Capacity_meq/100g     1000000 non-null float64
13  Electrical_Conductivity_dS/m          1000000 non-null float64
14  Soil_Color                             1000000 non-null object
15  Porosity_%                             1000000 non-null float64
16  Water_Holding_Capacity_%               1000000 non-null float64
dtypes: float64(14), int64(1), object(2)
memory usage: 129.7+ MB
```

```
In [50]: soil.head()
```

Out [50]:

	Soil_ID	Location_Latitude	Location_Longitude	Depth_cm	Texture	pH	Organic_Matter_%	Moisture_Content_%
0	1	-22.582779	34.256025	7.545292	sandy	6.774319	5.688127	5.24481
1	2	81.128575	-48.701829	12.457087	loamy	7.513037	2.386636	16.37623
2	3	41.758910	-178.064777	20.914292	sandy	5.312789	6.337647	27.75111
3	4	17.758527	21.991581	15.891773	sandy	8.866207	8.569403	30.57800
4	5	-61.916645	142.765348	10.514443	sandy	4.620486	1.060762	29.81580

5 rows × 22 columns

In [2]: *### Focus on Fire-Relevant Variables*

```
soil_fire = soil[[
    "Location_Latitude",
    "Location_Longitude",
    "Texture",
    "Moisture_Content_%",
    "Organic_Matter_%",
    "Porosity_%",
    "Water_Holding_Capacity_%",
    "Bulk_Density_g/cm³",
    "Cation_Exchange_Capacity_meq/100g"
]]
```

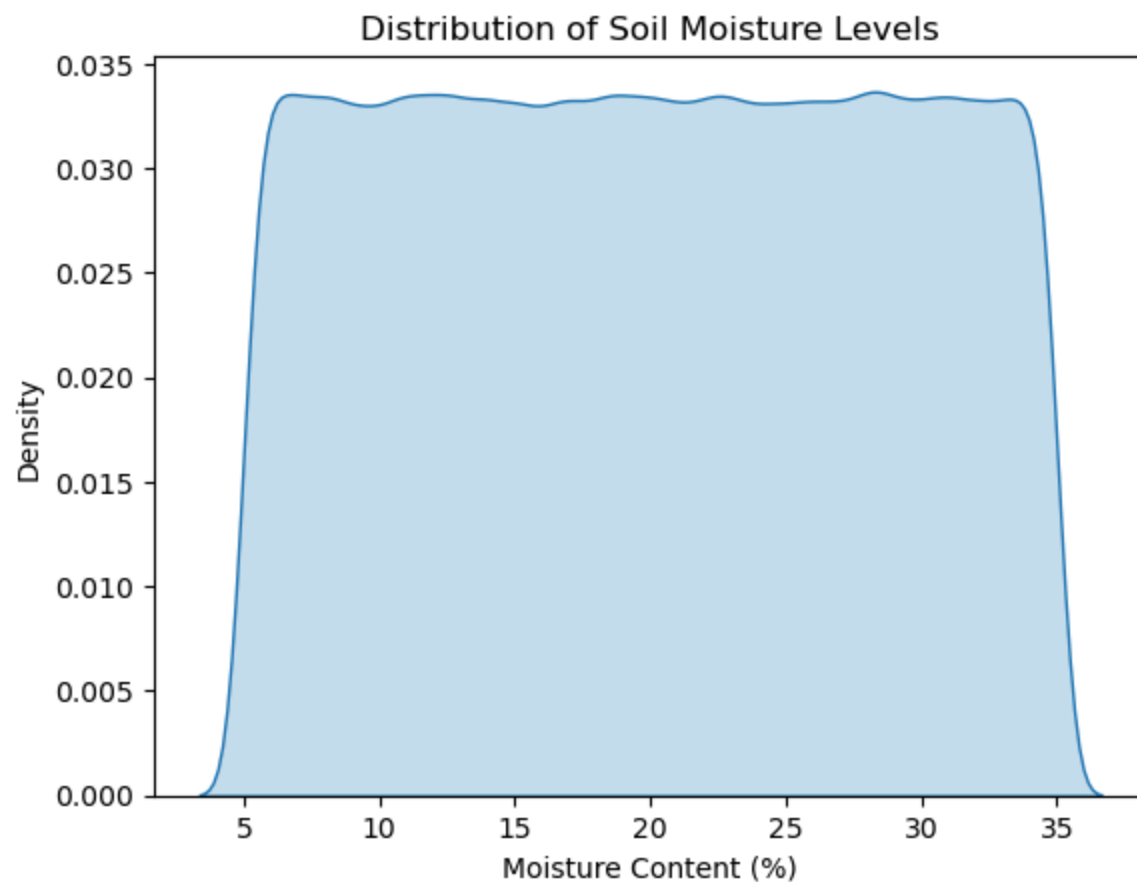
In []: area vs point ---- mapbox

In [3]: *### Visualizing Soil Moisture Distribution*
KDE Plot (Smooth Distribution Curve)

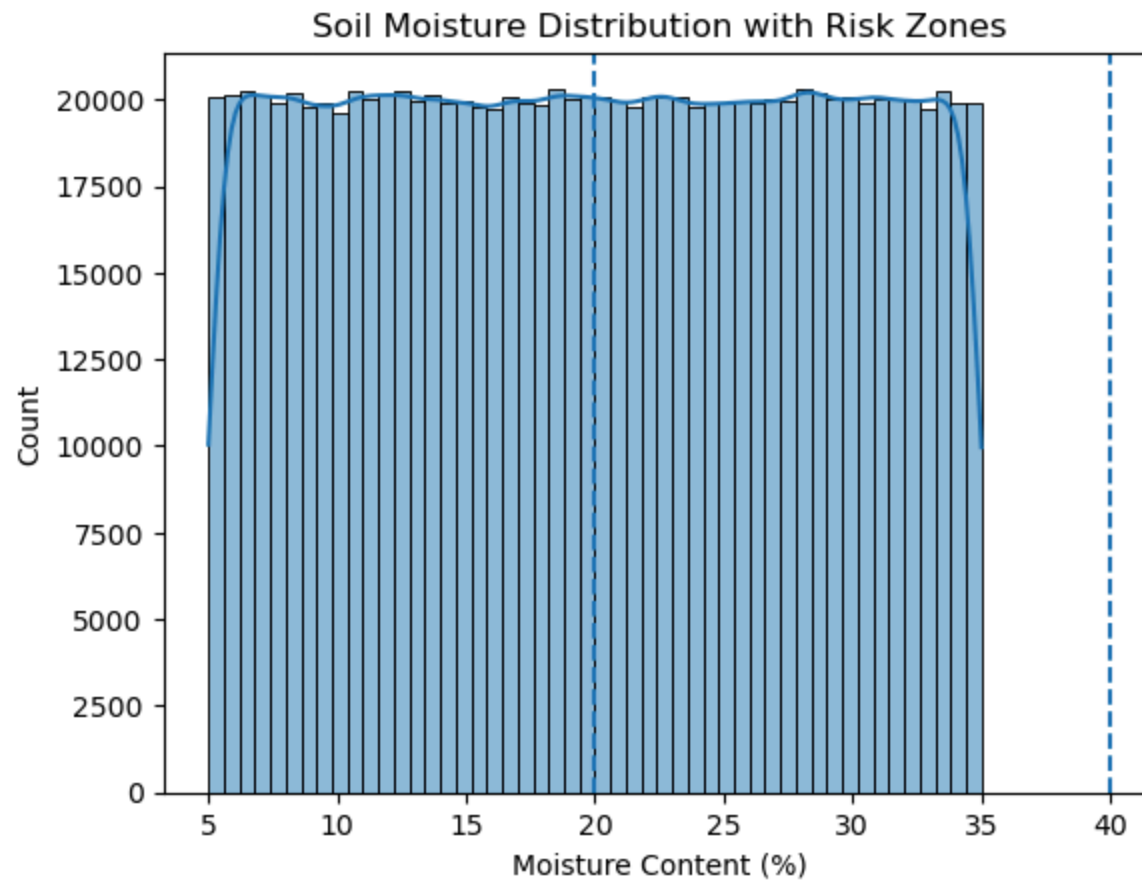
```
import seaborn as sns
import matplotlib.pyplot as plt

plt.figure()
sns.kdeplot(soil_fire["Moisture_Content_%"], fill=True)
plt.title("Distribution of Soil Moisture Levels")
plt.xlabel("Moisture Content (%)")
```

```
plt.ylabel("Density")  
plt.show()
```

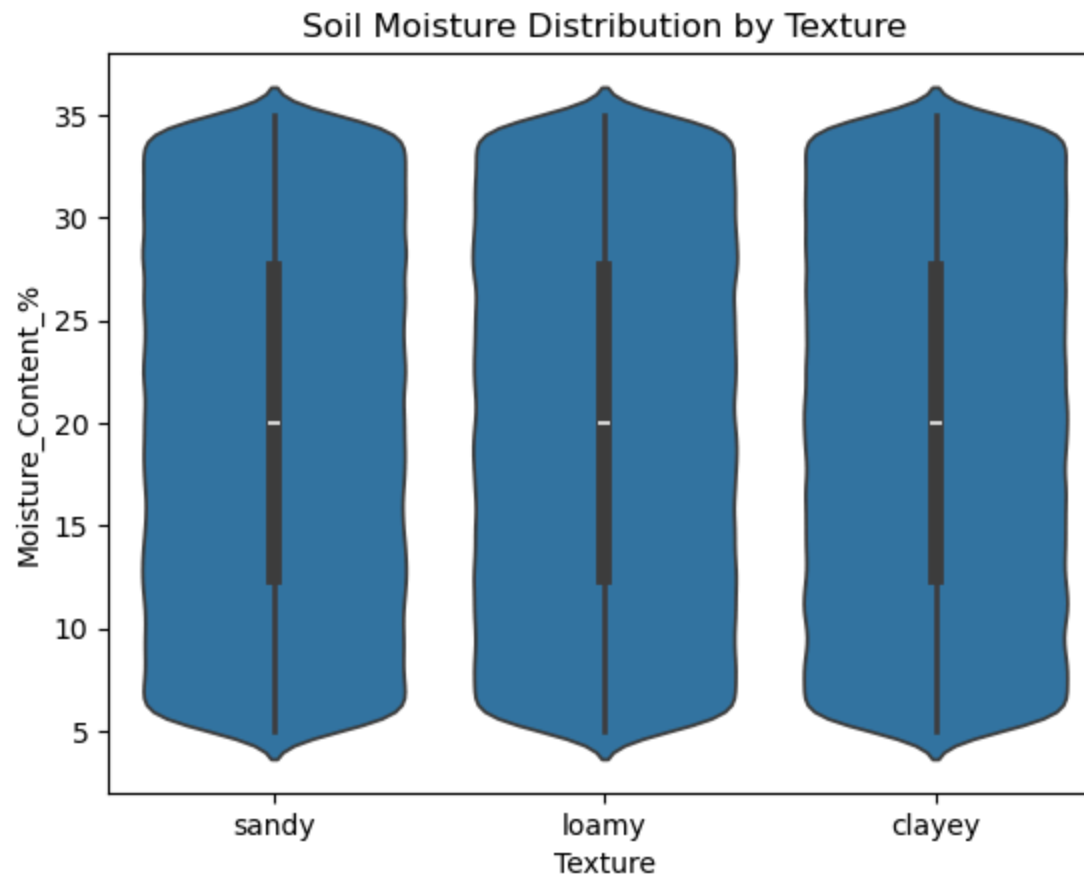


```
In [4]: ##### Moisture with Fire Risk Zones Highlighted  
plt.figure()  
sns.histplot(soil_fire["Moisture_Content_%"], bins=50, kde=True)  
  
plt.axvline(20, linestyle='--')  
plt.axvline(40, linestyle='--')  
  
plt.title("Soil Moisture Distribution with Risk Zones")  
plt.xlabel("Moisture Content (%)")  
plt.show()
```



```
In [5]: ### violin plot

plt.figure()
sns.violinplot(
    x="Texture",
    y="Moisture_Content_%",
    data=soil_fire
)
plt.title("Soil Moisture Distribution by Texture")
plt.show()
```



```
In [6]: ### Creating Soil Fire Risk

from sklearn.preprocessing import MinMaxScaler
import numpy as np

soil = soil.copy()

scaler = MinMaxScaler()

soil["moisture_scaled"] = scaler.fit_transform(soil[["Moisture_Content_%"]])
soil["whc_scaled"] = scaler.fit_transform(soil[["Water_Holding_Capacity_%"]])
soil["organic_scaled"] = scaler.fit_transform(soil[["Organic_Matter_%"]])

# Dryness score (higher = more fire risk)
```

```
soil["dryness_score"] = (  
    (1 - soil["moisture_scaled"]) * 0.5 +  
    (1 - soil["whc_scaled"]) * 0.3 +  
    (1 - soil["organic_scaled"]) * 0.2  
)
```

In [7]: *#### Convert risk levels*

```
soil["soil_fire_risk"] = pd.qcut(  
    soil["dryness_score"],  
    q=3,  
    labels=["Low", "Medium", "High"]  
)
```

In [8]: *#### Create global grid*

```
soil["lat_bin"] = soil["Location_Latitude"].round(1)  
soil["lon_bin"] = soil["Location_Longitude"].round(1)  
  
grid = (  
    soil.groupby(["lat_bin", "lon_bin"])["dryness_score"]  
        .mean()  
        .reset_index()  
)
```

In [9]: *#### classify grid level risks*

```
grid["grid_risk"] = pd.qcut(  
    grid["dryness_score"],  
    q=3,  
    labels=["Low", "Medium", "High"]  
)
```

In [10]: *### full globe visulaization*

```
import plotly.express as px  
  
color_map = {  
    "Low": "green",  
    "Medium": "yellow",  
    "High": "red"  
}  
  
fig = px.scatter_geo(  
    #
```

```
grid,
lat="lat_bin",
lon="lon_bin",
color="grid_risk",
color_discrete_map=color_map,
category_orders={"grid_risk": ["Low", "Medium", "High"]},
title="Global Soil-Based Fire Risk Map (Green → Yellow → Red)"
)

fig.update_traces(marker=dict(size=6, opacity=0.8))

fig.update_layout(
    height=700,
    geo=dict(
        projection_type="natural earth",
        showland=True,
        landcolor="rgb(240,240,240)",
        showocean=True,
        oceancolor="rgb(220,230,250)"
    )
)

fig.show()
```

Global Soil-Based Fire Risk Map (Green → Yellow → Red)



```
In [11]: fig = px.scatter_geo(
    soil.sample(50000),
    lat="Location_Latitude",
    lon="Location_Longitude",
    color="soil_fire_risk",
    color_discrete_map=color_map,
```



```
    projection="orthographic",  
    title="3D Globe View – Soil Fire Risk"  
)  
  
fig.update_traces(marker=dict(size=3))  
fig.update_layout(height=700)  
  
fig.show()
```

3D Globe View – Soil Fire Risk



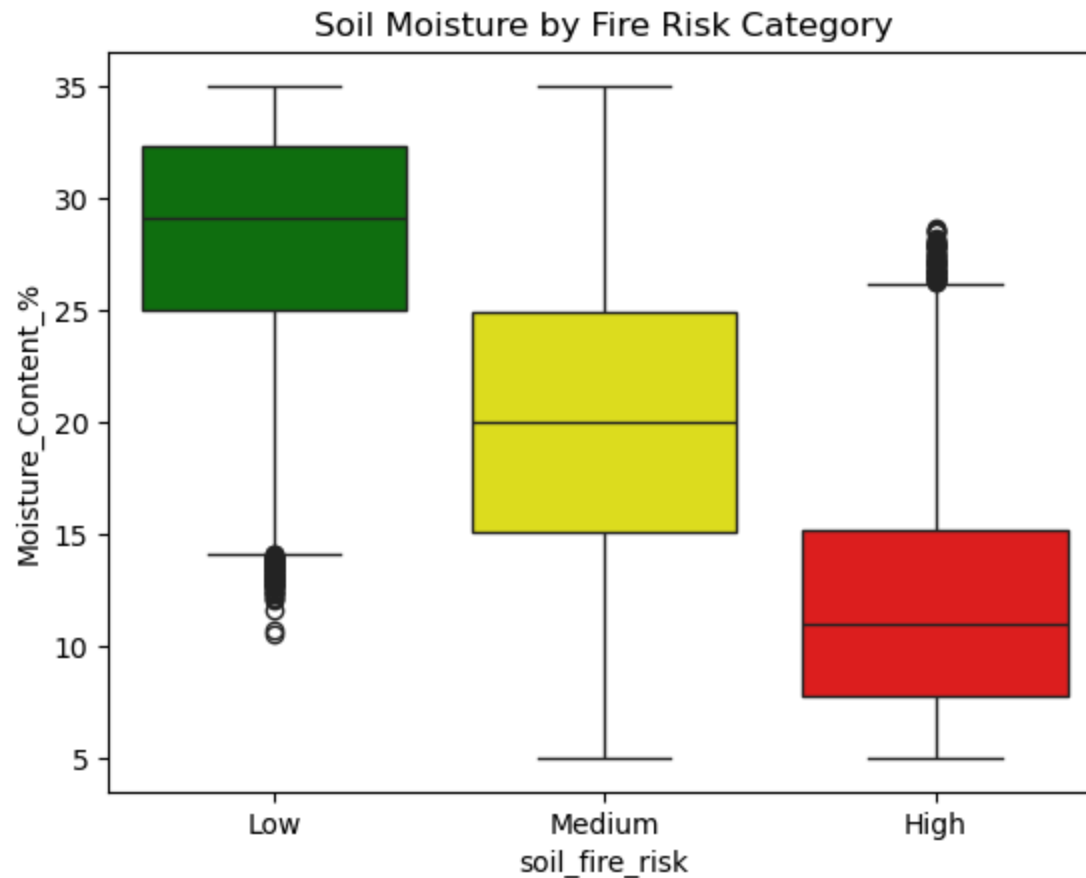
```
In [12]: import seaborn as sns
import matplotlib.pyplot as plt

sns.boxplot(
    x="soil_fire_risk",
    y="Moisture_Content_%",
```

```
data=soil.sample(50000),  
palette={"Low":"green","Medium":"yellow","High":"red"}  
)  
  
plt.title("Soil Moisture by Fire Risk Category")  
plt.show()
```

/tmp/ipykernel_2245/3261273911.py:4: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.



```
In [13]: soil["soil_fire_risk"].value_counts()
```

```
Out[13]: soil_fire_risk
Low      333334
High     333334
Medium   333332
Name: count, dtype: int64
```

```
In [14]: soil["soil_fire_risk"].isna().sum()
```

```
Out[14]: 0
```

```
In [15]: soil["soil_fire_risk"] = soil["soil_fire_risk"].astype(str)
```

```
In [16]: import plotly.express as px

sample_df = soil.sample(50000).copy()

color_map = {
    "Low": "green",
    "Medium": "yellow",
    "High": "red"
}

fig = px.scatter_geo(
    sample_df,
    lat="Location_Latitude",
    lon="Location_Longitude",
    color="soil_fire_risk",
    projection="natural earth",
    color_discrete_map=color_map,
    title="Global Soil Fire Risk Map"
)

fig.update_traces(marker=dict(size=4, opacity=0.8))

fig.update_layout(
    height=700,
    geo=dict(
        showland=True,
        landcolor="lightgray",
```

```
    showocean=True,  
    oceancolor="lightblue"  
)  
)  
fig.show()
```

Global Soil Fire Risk Map



```
In [17]: import pandas as pd
import numpy as np
import plotly.express as px
import matplotlib.pyplot as plt
```

```
In [18]: soil = pd.read_csv("soil_data.csv")
```

```
In [19]: viirs_noaa20 = pd.read_csv("fire_nrt_J1V-C2_565335.csv")
viirs_snpp = pd.read_csv("fire_nrt_SV-C2_565336.csv")
modis = pd.read_csv("fire_nrt_M-C61_565334.csv")
```

```
In [20]: viirs_noaa20["source"] = "VIIRS_NOAA20"
viirs_snpp["source"] = "VIIRS_SNPP"
modis["source"] = "MODIS"
```

```
In [21]: fire = pd.concat([viirs_noaa20, viirs_snpp, modis], ignore_index=True)
```

```
In [22]: ### Overlay Soil Risk vs Real Fires (VALIDATION)
from sklearn.preprocessing import MinMaxScaler

scaler = MinMaxScaler()

soil["moisture_norm"] = scaler.fit_transform(soil[["Moisture_Content_%"]])
soil["climate_risk"] = 1 - (abs(soil["Location_Latitude"]) / 90)

soil["combined_risk"] = (
    (1 - soil["moisture_norm"]) * 0.6 +
    soil["climate_risk"] * 0.4
)
```

```
In [23]: ### Create Same 5° Grid for Soil & Fire
# Soil grid
soil["lat_block"] = (soil["Location_Latitude"] // 5) * 5
soil["lon_block"] = (soil["Location_Longitude"] // 5) * 5

soil_grid = (
    soil.groupby(["lat_block", "lon_block"])["combined_risk"]
    .mean()
    .reset_index()
```

```
)  
  
# Fire grid  
fire["lat_block"] = (fire["latitude"] // 5) * 5  
fire["lon_block"] = (fire["longitude"] // 5) * 5  
  
fire_grid = (  
    fire.groupby(["lat_block", "lon_block"])  
        .size()  
        .reset_index(name="fire_count")  
)
```

```
In [24]: ##### merge soil + fire  
merged = pd.merge(  
    soil_grid,  
    fire_grid,  
    on=["lat_block", "lon_block"],  
    how="left"  
)  
  
merged["fire_count"] = merged["fire_count"].fillna(0)
```

```
In [25]: ##### correlation  
correlation = merged["combined_risk"].corr(merged["fire_count"])  
print("Correlation between soil risk and fire occurrence:", correlation)
```

Correlation between soil risk and fire occurrence: 0.16433566055838839

```
In [26]: ##### create binary fire label  
merged["fire_label"] = np.where(merged["fire_count"] > 0, 1, 0)
```

```
In [27]: ##### train ML model  
from sklearn.model_selection import train_test_split  
from sklearn.ensemble import RandomForestClassifier  
from sklearn.metrics import classification_report  
  
X = merged[["combined_risk"]]  
y = merged["fire_label"]  
  
X_train, X_test, y_train, y_test = train_test_split(  
    X, y, test_size=0.3, random_state=42
```

```
)

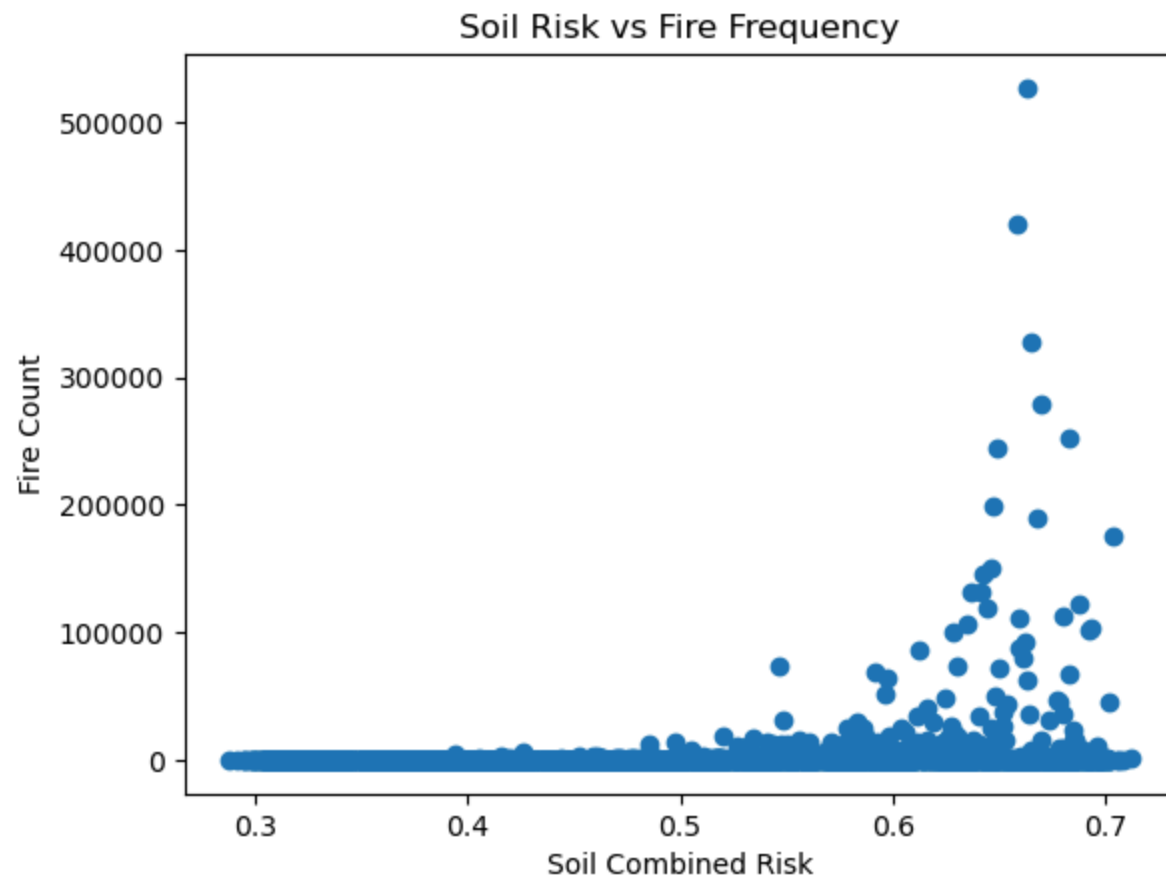
model = RandomForestClassifier()
model.fit(X_train, y_train)

pred = model.predict(X_test)

print(classification_report(y_test, pred))
```

	precision	recall	f1-score	support
0	0.76	0.74	0.75	551
1	0.41	0.44	0.42	227
accuracy			0.65	778
macro avg	0.58	0.59	0.59	778
weighted avg	0.66	0.65	0.65	778

```
In [28]: ### risk dist comparision
plt.figure()
plt.scatter(merged["combined_risk"], merged["fire_count"])
plt.xlabel("Soil Combined Risk")
plt.ylabel("Fire Count")
plt.title("Soil Risk vs Fire Frequency")
plt.show()
```

```
In [29]: ### early warnign threshold ---- this says How many real fires occurred in high soil-risk zones?
merged["early_warning"] = np.where(
    merged["combined_risk"] > 0.7,
    1,
    0
)

print("Percentage of fires in early warning zones:",
      merged[merged["fire_label"] == 1]["early_warning"].mean())
```

Percentage of fires in early warning zones: 0.010568031704095112

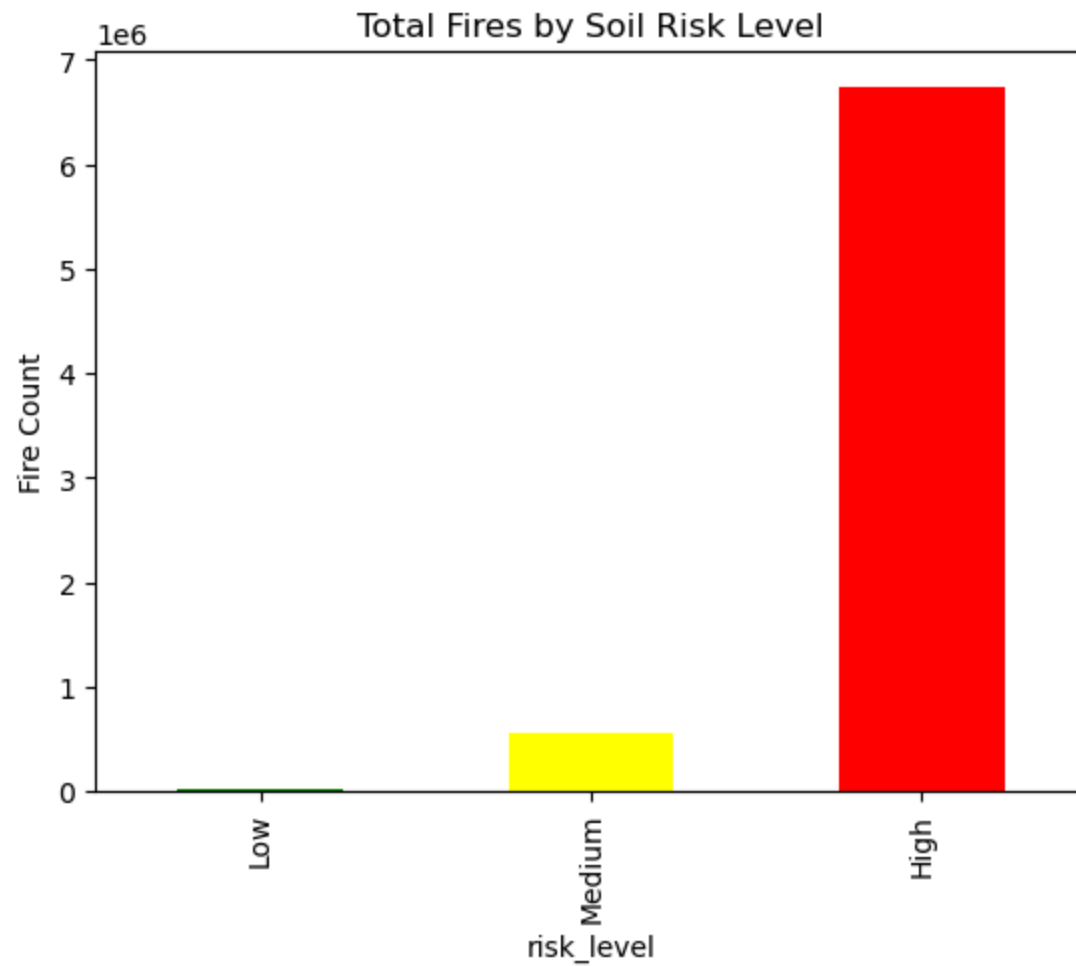
```
In [30]: ##### Do high soil-risk zones actually have more fires?
merged.groupby("fire_label")["combined_risk"].mean()
```

```
Out[30]: fire_label  
0      0.472619  
1      0.566727  
Name: combined_risk, dtype: float64
```

```
In [31]: ##### compare fire count by risk level  
merged["risk_level"] = pd.qcut(  
    merged["combined_risk"],  
    q=3,  
    labels=["Low", "Medium", "High"]  
)  
  
fire_distribution = merged.groupby("risk_level")["fire_count"].sum()  
fire_distribution.plot(kind="bar", color=["green", "yellow", "red"])  
plt.title("Total Fires by Soil Risk Level")  
plt.ylabel("Fire Count")  
plt.show()
```

/tmp/ipykernel_2245/1717269948.py:8: FutureWarning:

The default of observed=False is deprecated and will be changed to True in a future version of pandas. Pass observed=False to retain current behavior or observed=True to adopt the future default and silence this warning.



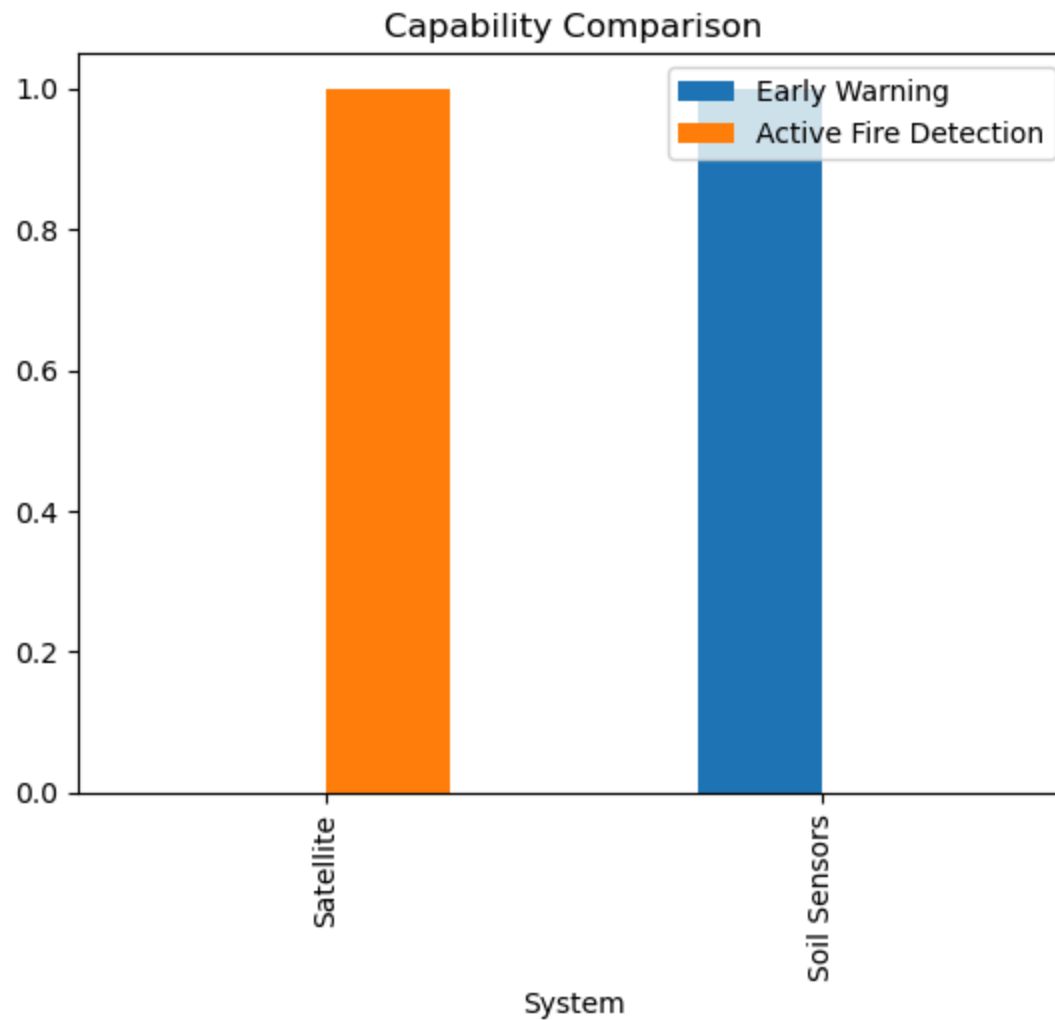
```
In [35]: from sklearn.metrics import roc_auc_score

prob = model.predict_proba(X_test)[:,-1]
roc_auc_score(y_test, prob)
### if 0.7 → soil model reasonably predictive
### ~0.5 → random
### 0.8 → strong
```

```
Out[35]: 0.6789577620185965
```

```
In [36]: ##### Prevention vs Detection Chart
comparison = pd.DataFrame({
    "System":["Satellite","Soil Sensors"],
    "Early Warning":[0,1],
    "Active Fire Detection":[1,0]
})

comparison.set_index("System").plot(kind="bar")
plt.title("Capability Comparison")
plt.show()
```



```
In [37]: ### Risk Zone Accuracy Gauge  
high_risk_fire_percent = (  
    merged[merged["fire_label"]==1]["risk_level"=="High"]  
).mean()*100  
  
print("Percent of Fires in High Soil Risk Zones:", high_risk_fire_percent)
```

Percent of Fires in High Soil Risk Zones: 53.5006605019815

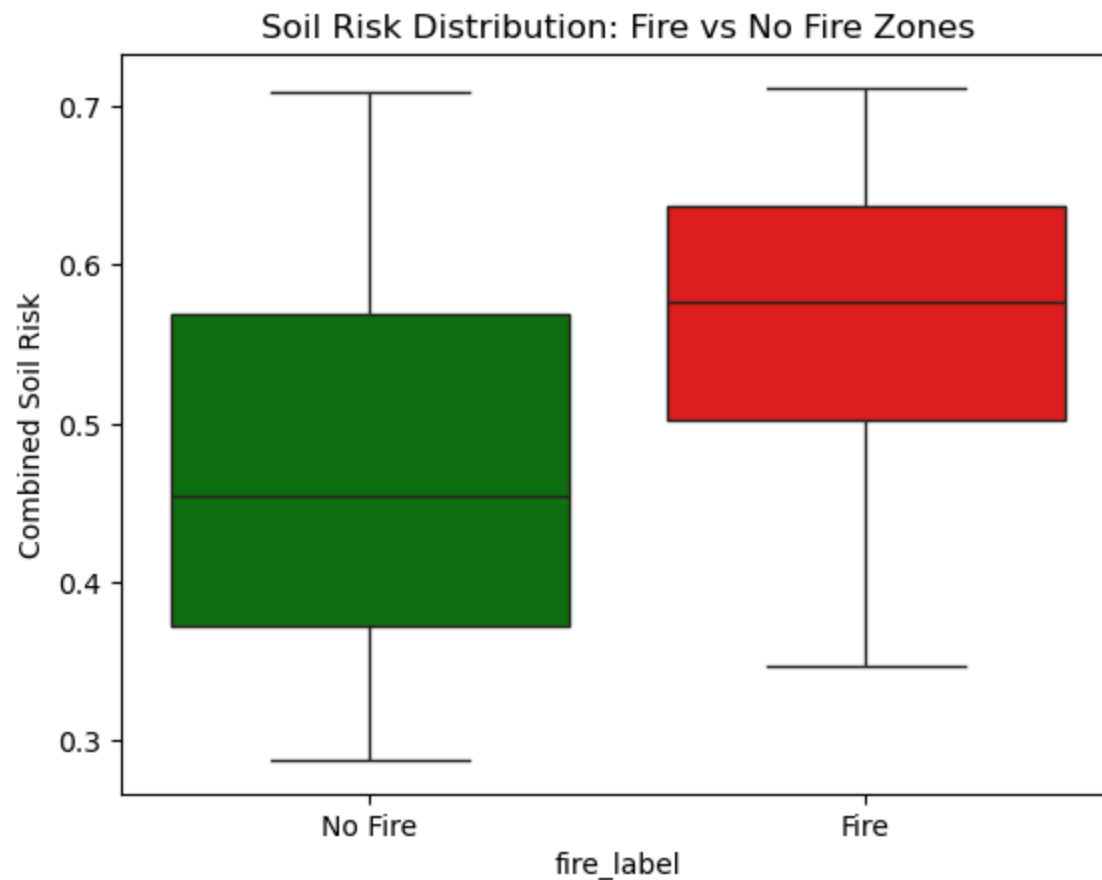
```
In [38]: ##### Compare Average Soil Risk (Fire vs No Fire)
import matplotlib.pyplot as plt
import seaborn as sns

sns.boxplot(
    x="fire_label",
    y="combined_risk",
    data=merged,
    palette=["green", "red"]
)

plt.xticks([0,1], ["No Fire", "Fire"])
plt.title("Soil Risk Distribution: Fire vs No Fire Zones")
plt.ylabel("Combined Soil Risk")
plt.show()
```

/tmp/ipykernel_2245/1000842811.py:5: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.



```
In [41]: ##### Percentage of Fires in High-Risk Zones (Executive Metric)
high_fire_percent = (
    merged[merged["fire_label"]==1]["risk_level"]=="High"
).mean()*100

print("Percentage of Fires in High Soil Risk Zones:",
      round(high_fire_percent,2),"%")

##### if 60% → soil sensors useful predictor
##### if < 40% → weak predictor
```

Percentage of Fires in High Soil Risk Zones: 53.5 %

```
In [42]: ### Visual Early Warning vs Detection Comparison
comparison = {
```

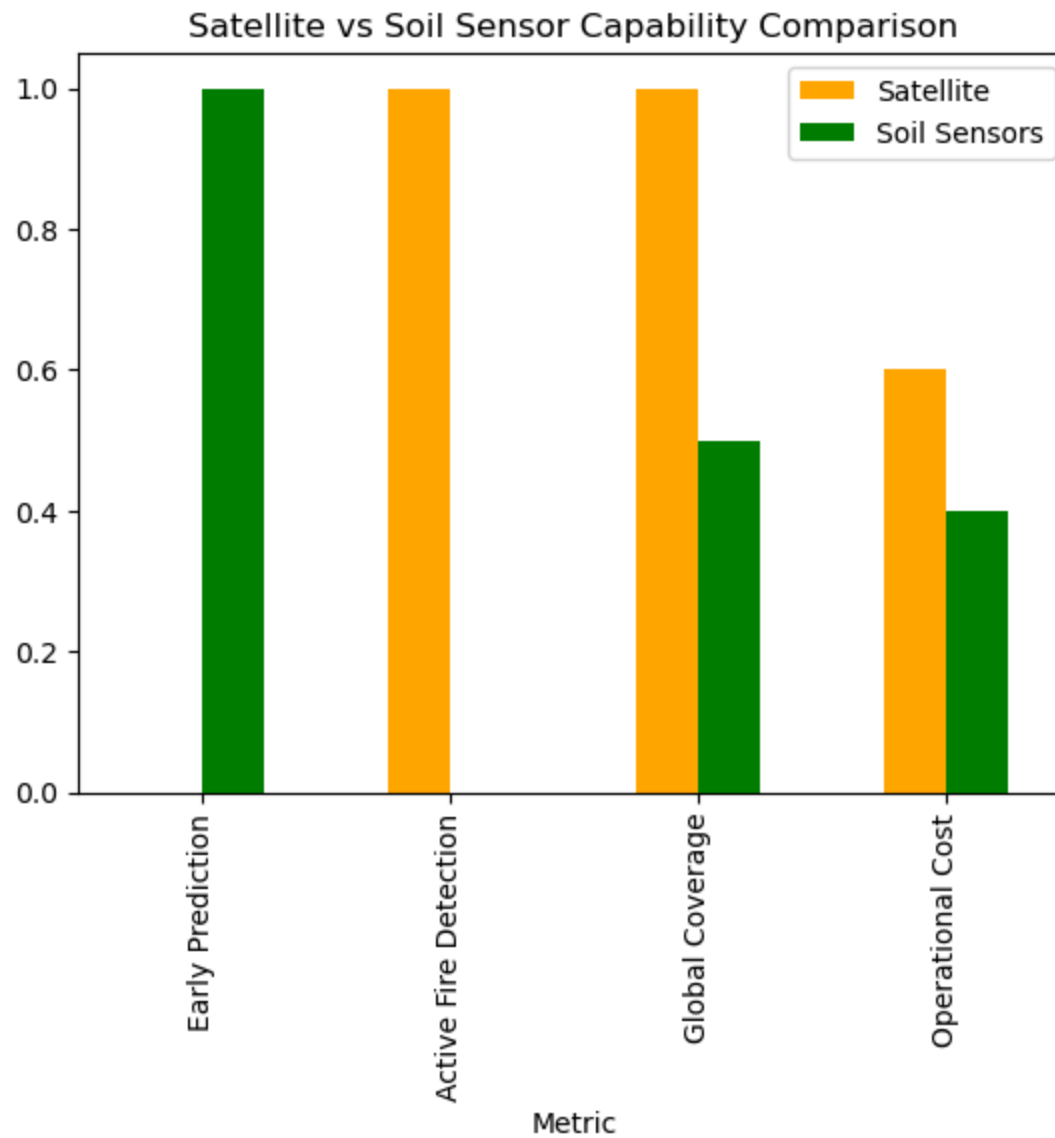
```
"Metric": [
    "Early Prediction",
    "Active Fire Detection",
    "Global Coverage",
    "Operational Cost"
],
"Satellite": [0,1,1,0.6],
"Soil Sensors": [1,0,0.5,0.4]
}

import pandas as pd

df_compare = pd.DataFrame(comparison)

df_compare.set_index("Metric").plot(
    kind="bar",
    color=["orange", "green"]
)

plt.title("Satellite vs Soil Sensor Capability Comparison")
plt.show()
```

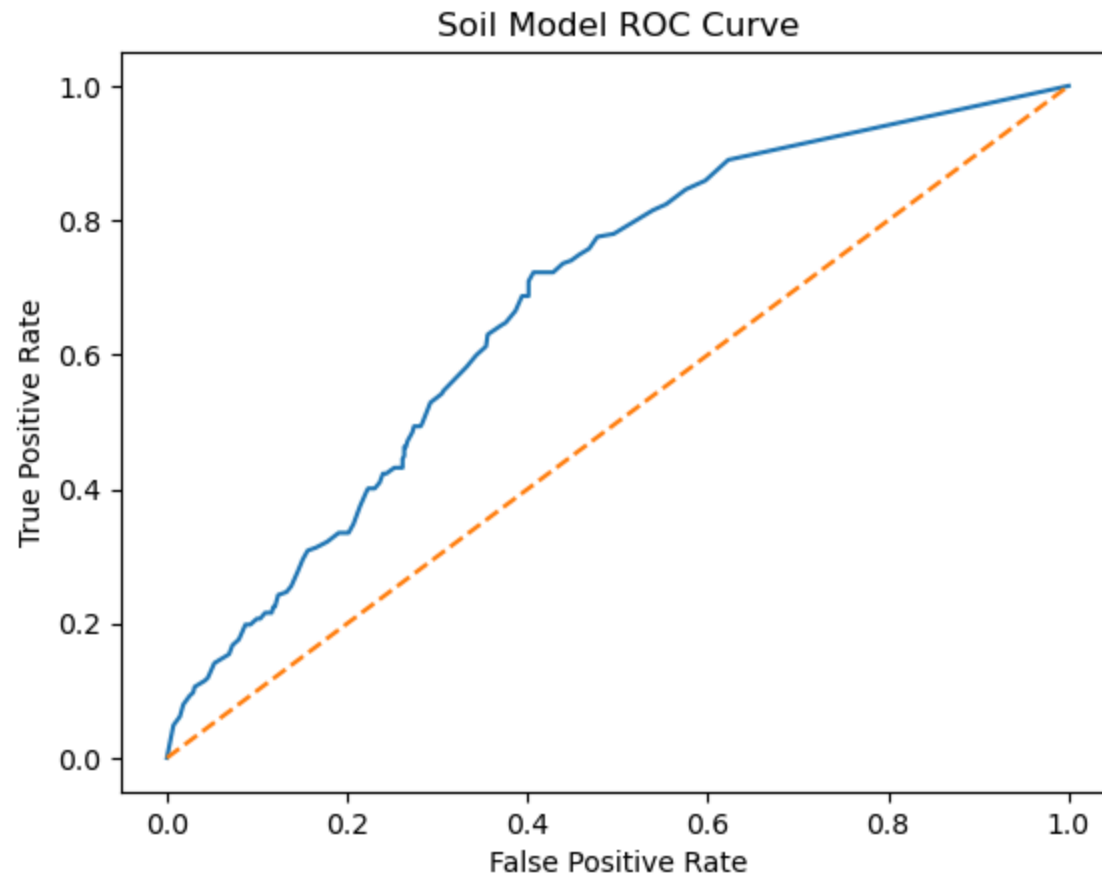
```
In [44]: ### ROC Curve (Predictive Strength)
from sklearn.metrics import roc_curve, roc_auc_score

prob = model.predict_proba(X_test)[:,-1]
fpr, tpr, _ = roc_curve(y_test, prob)
```

```
plt.plot(fpr, tpr)
plt.plot([0,1],[0,1], linestyle="--")
plt.title("Soil Model ROC Curve")
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.show()

print("ROC-AUC Score:", roc_auc_score(y_test, prob))

##### if 0.7 → soil model has predictive value
##### if ~0.5 → no predictive strength
```



ROC-AUC Score: 0.6789577620185965

```
In [45]: fig = px.scatter_geo(
    merged,
    lat="lat_block",
    lon="lon_block",
    size="fire_count",
    color="combined_risk",
    projection="natural earth",
    color_continuous_scale=[
        [0.0,"green"],
        [0.5,"yellow"],
        [1.0,"red"]
    ],
    title="Soil Risk (Color) vs Satellite Fire Detection (Size)"
)

fig.update_layout(height=750)
fig.show()
```

Soil Risk (Color) vs Satellite Fire Detection (Size)



```
In [46]: merged["fire_count"].describe()
```

```
Out [46]: count      2592.000000
mean      2821.050926
std       20940.167501
min        0.000000
25%        0.000000
50%        0.000000
75%       15.000000
max      527321.000000
Name: fire_count, dtype: float64
```

```
In [47]: import numpy as np

merged["fire_log"] = np.log1p(merged["fire_count"])
```

```
In [48]: merged["risk_norm"] = (
    merged["combined_risk"] - merged["combined_risk"].min()
) / (
    merged["combined_risk"].max() - merged["combined_risk"].min()
)
```

```
In [49]: import plotly.express as px

fig = px.scatter_geo(
    merged,
    lat="lat_block",
    lon="lon_block",
    size="fire_log",
    color="risk_norm",
    projection="natural earth",
    range_color=(0,1),
    color_continuous_scale=[
        [0.0, "green"],
        [0.5, "yellow"],
        [1.0, "red"]
    ],
    title="Soil Risk (Color) vs Satellite Fires (Log-Scaled Size)"
)

fig.update_traces(marker=dict(opacity=0.85))
fig.update_layout(height=750)
```

```
fig.show()
```

Soil Risk (Color) vs Satellite Fires (Log-Scaled Size)



In []: